

SCHOOL OF COMPUTER SCIENCE ENGINEERING AND INFORMATION SYSTEMS



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

ISWE309L – Cloud Computing

Slot: A1 + TA1

Course Project Review-2

Project Title: CloudConfig Ops – Serverless Cloud Infrastructure Configuration & Compliance Platform using Mercurial

Faculty: Prof. Bhuvaneshwari M.S.

Project Team

Reg. No.	Student Name	Email	Contact no.
24MIS0206	Harsh Mishra	harsh.mishra2024@vitstudent.ac.in	9125857000
24MIS0177	Ansh Dubey	ansh.dubey2024@vitstudent.ac.in	9406138492

1. Introduction

- **Background of the project:** Modern software applications are almost universally hosted across cloud platforms like Amazon Web Services (AWS). Every cloud deployment relies heavily on configuration files such as environment variables (.env), reverse-proxy configurations (nginx.conf), and container manifests (docker-compose.yml). These files govern application behavior, network rules, and database connections across Development, Testing, and Production environments.
- **Problem statement:** System administrators and developers often modify cloud configuration files directly on live virtual machines to resolve immediate issues. This manual process bypasses standard review procedures and results in “Configuration Drift”—a critical state where the live cloud environment diverges from the documented baseline. Unchecked drift causes silent deployment failures and extended troubleshooting times.
- **Need for the project:** Treating cloud infrastructure settings as unmanaged text files creates serious operational and compliance risks. Modern governance frameworks require verifiable audit trails that prove who made changes to cloud systems, when they were made, and who authorized them. Ad-hoc, manual server updates cannot provide this level of traceability.
- **Motivation:** Commercial Infrastructure-as-Code (IaC) tools primarily focus on provisioning physical cloud resources (such as virtual networks and disks), which is overkill for managing simple application configuration files. There is a distinct gap for a lightweight, cloud-hosted governance platform that treats server configuration files as version-controlled artifacts governed by strict review workflows.
- **Scope:** CloudConfig Ops provides a web-based, cloud-hosted control plane on AWS to manage configuration items. The system provides automated version control using Mercurial SCM, a gated change-request workflow, serverless configuration drift detection using AWS Lambda, and fast baseline rollbacks via a custom Python SSH deployment engine.

2. Existing System

Already Existing System

1. Thiagarajan, G., Bist, V., & Nayak, P. (2024). AI-Driven Configuration Drift Detection in Cloud Environments. International Journal of Communication Networks and Information Security, 16(5), 721–743.

- *i. Description:* Compares live AWS states with IaC baselines using ML to flag resource drift.
- *ii. Technology used:* Machine learning models, Terraform, AWS Config logs.
- *iii. Advantages:* Reduces dependence on manual monitoring and scales across dynamic environments.
- *iv. Limitations:* Operates at the virtual resource level rather than tracking individual application configuration files.

2. Harikrishnan, A. (2026). Infrastructure as Code (IaC) Drift Detection: State Management Using Event-Driven Architecture. Journal of Computational Analysis and Applications, 35(3), 10–23.

- *i. Description:* Proposes an event-driven mechanism to detect infrastructure state changes in real time.
- *ii. Technology used:* Event-driven architecture, IaC state management tools.
- *iii. Advantages:* Near real-time drift detection, removing polling overhead.
- *iv. Limitations:* Lacks human-in-the-loop review workflows for specific configuration items.

3. Verdet, A., Hamdaqa, M., Da Silva, L. M. P., & Khomh, F. (2023). Exploring Security Practices in Infrastructure as Code: An Empirical Study. arXiv:2308.03952 [cs.CR].

- *i. Description:* Analyzes security flaws across 401 open-source Terraform repositories.
- *ii. Technology used:* Terraform, AWS, Azure, GCP.
- *iii. Advantages:* Large-scale, evidence-based picture of common IaC security gaps.
- *iv. Limitations:* Purely empirical study; provides no active governance or configuration control system.

4. Cherukuri, B. R. (2024). Comparative Analysis of Docker and Kubernetes for Cloud Resource and Microservice Management. International Journal of Science and Research Archive, 13(1), 3302–3315.

- *i. Description:* Evaluates Docker and Kubernetes for microservice deployment and configuration.
- *ii. Technology used:* Docker, Kubernetes.
- *iii. Advantages:* Clarifies optimal orchestration tools for varied deployment scenarios.
- *iv. Limitations:* Descriptive overview without revision history or audit trail mechanisms.

5. Kumar, A., Nadeem, M., & Shameem, M. (2025). A Systematic Literature Review for Investigating DevOps Metrics to Implement in Software Development Organizations. Journal of Software: Evolution and Process, 37(1), e2733.

- *i. Description:* Consolidates 32 metrics to assess DevOps process and pipeline maturity.
- *ii. Technology used:* General CI/CD and DevOps toolchains.
- *iii. Advantages:* Provides a consolidated metric set for evaluating pipeline health.
- *iv. Limitations:* Focuses on measurement rather than providing a technical configuration platform.

6. Babalola, O., Nwatu, C. E., Folorunso, A., & Adewa, A. (2024). A Governance Framework Model for Cloud Computing: Role of AI, Security, Compliance, and

Management. World Journal of Advanced Research and Reviews, 24(2), 1969–1982.

- *i. Description:* Introduces a conceptual multi-cloud governance and compliance framework.
- *ii. Technology used:* AI-based monitoring, compliance-checking models.
- *iii. Advantages:* Holistic governance view spanning multiple regulatory frameworks.
- *iv. Limitations:* High-level framework without an implementation engine for configuration files.

7. Jayaram, Y. (2023). Data Governance and Content Lifecycle Automation in the Cloud for Secure, Compliance-Oriented Data Operations. International Journal of AI, BigData, Computational and Management Studies, 4(3), 124–133.

- *i. Description:* Automates cloud data classification and compliance tracking.
- *ii. Technology used:* Rule engines, workflow orchestration, encryption automation.
- *iii. Advantages:* Reduces compliance risk and improves audit readiness.
- *iv. Limitations:* Targets stored application data rather than deployment configuration artifacts.

8. Olabanji, S. O., Olaniyi, O. O., Adigwe, C. S., Okunleye, O. J., & Oladoyinbo, T. O. (2024). AI for Identity and Access Management (IAM) in the Cloud. Asian Journal of Research in Computer Science, 17(3), 38–56.

- *i. Description:* Uses AI models to evaluate authorization patterns in cloud IAM systems.
- *ii. Technology used:* AI/ML-based authentication and role-based access control.
- *iii. Advantages:* Improves detection of anomalous or unauthorized access patterns.
- *iv. Limitations:* Restricted to IAM identity; does not address configuration lifecycle management.

9. Saleh, S. M., Madhavji, N., & Steinbacher, J. (2024). A Systematic Literature Review on Continuous Integration and Deployment (CI/CD) for Secure Cloud Computing. Proceedings of the 20th International Conference on Web Information Systems and Technologies, 331–341.

- *i. Description:* Synthesizes best practices for pipeline security across cloud environments.
- *ii. Technology used:* Harbor, SonarQube, GitHub Actions.
- *iii. Advantages:* Broad view of security gaps across the CI/CD lifecycle.
- *iv. Limitations:* Survey paper; does not deliver an integrated configuration baseline tool.

10. Ganesan, P. (2024). Cloud-Based Disaster Recovery: Reducing Risk and Improving Continuity. Journal of Artificial Intelligence & Cloud Computing, 3(1), E162.

- *i. Description:* Analyzes cloud failover and storage replication techniques.
- *ii. Technology used:* Amazon S3 Cross-Region Replication, Azure Geo-Redundant Storage.
- *iii. Advantages:* Improves recovery time objectives and business continuity.
- *iv. Limitations:* Focuses on infrastructure-level disaster recovery rather than granular configuration rollback.

11. Winkler, F., & Westner, M. (2023). A Systematic Literature Review of DevOps Success Factors and Adoption Models. Proceedings of the 12th International Symposium on Information and Communication Technology, 1–8.

- *i. Description:* Identifies organizational factors governing DevOps transitions.
- *ii. Technology used:* DevOps culture and process-maturity models.
- *iii. Advantages:* Useful for planning organization-wide DevOps transitions.
- *iv. Limitations:* Organizational and process-focused; provides no software solution.

12. Alharthi, S., Alshamsi, A., Alseiari, A., & Alwarafy, A. (2024). Auto-Scaling Techniques in Cloud Computing: Issues and Research Directions. Sensors, 24(17), 5551.

- *i. Description:* Evaluates auto-scaling algorithms for cloud resource elasticity.

- *ii. Technology used:* Threshold-based rules, queuing theory, ML forecasting.
- *iii. Advantages:* Improves cost-efficiency and performance under variable workload.
- *iv. Limitations:* Concentrates purely on load scaling rather than configuration integrity.

13. Deochake, S. (2024). ABACUS: A FinOps Service for Cloud Cost Optimization. arXiv:2501.14753 [cs.DC].

- *i. Description:* Optimizes cloud billing by analyzing resource utilization patterns.
- *ii. Technology used:* Cloud billing APIs, usage-analysis heuristics.
- *iii. Advantages:* Directly supports cost-effectiveness by identifying idle resources.
- *iv. Limitations:* Focuses entirely on FinOps without configuration change tracking.

14. Essien, I. A., et al. (2023). Integrated Governance, Risk, and Compliance Framework for Multi-Cloud Security and Global Regulatory Alignment. IRE Journals, 7(6).

- *i. Description:* Multi-cloud policy mapping framework for regulatory compliance.
- *ii. Technology used:* Multi-cloud policy engines, compliance-mapping tools.
- *iii. Advantages:* Addresses regulatory alignment across heterogeneous cloud estates.
- *iv. Limitations:* Conceptual architecture lacking direct integration with version control systems.

15. Nangi, P. R., & Obannagari, C. K. R. (2025). AI-Driven Security Automation for Continuous Compliance Monitoring in Regulated Cloud Environments. International Journal of Emerging Trends in Computer Science and Information Technology, 6(2).

- *i. Description:* Uses NLP and ML to enforce security compliance rules across cloud infrastructure.
- *ii. Technology used:* NLP, ML-based anomaly detection, automated remediation.
- *iii. Advantages:* Reduces policy violations and response times.
- *iv. Limitations:* Enterprise compliance monitoring; lacks granular file-level approval mechanisms.

Gaps identified

- Most cloud drift solutions monitor compute-level resources (CPU, VMs) while ignoring application configuration files.
- Current configuration workflows lack an integrated, reviewer-gated approval pipeline directly connected to a distributed version control system.
- Existing compliance tools require running background agents 24/7, causing high compute overhead, rather than utilizing cost-effective serverless event-driven architectures.
- Existing literature relies almost exclusively on Git ecosystems; the use of Mercurial SCM for automated cloud configuration governance remains unexplored.
- There is a missing link between configuration baselines and automated, one-click rollback mechanisms for specific files during cloud deployment failures.

3. Objectives of the Project

- Build a centralized, cloud-hosted web platform to manage deployment configuration files across Development, Testing, and Production cloud environments.
- Use Mercurial SCM as a dedicated backend version control engine to track every configuration change with complete commit history.
- Implement a structured Change Request and Approval workflow that prevents unauthorized configuration changes from reaching cloud servers.
- Develop a custom Python-driven deployment engine to update remote cloud instances securely over SSH.

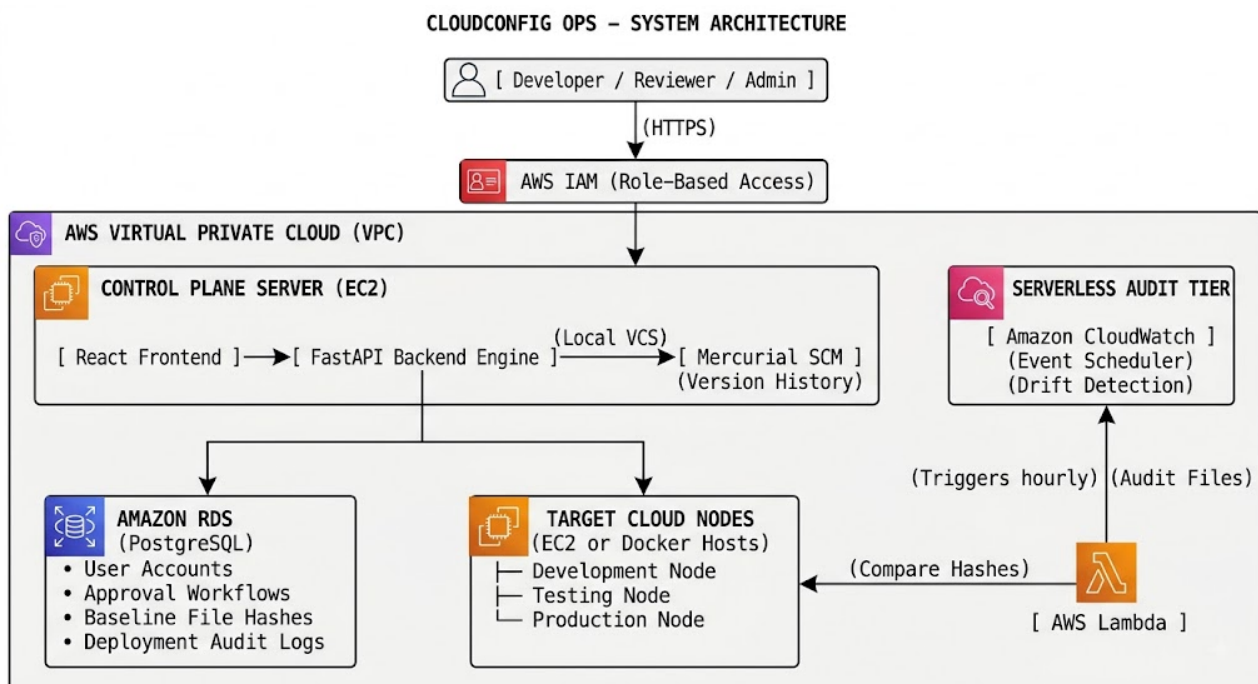
- Implement a serverless Drift Detection Engine using AWS Lambda and Amazon CloudWatch to verify file checksums and detect unauthorized server modifications.
- Enable one-click rollback to restore target cloud servers to previously verified baselines during deployment failures.

4. Abstract

Modern cloud environments rely on numerous application configuration files, including environment variables, proxy configurations, and container manifests, which are frequently modified manually. This practice causes configuration drift, environment inconsistencies, deployment failures, and security compliance violations. This project proposes CloudConfig Ops, a cloud-native configuration management platform hosted on Amazon Web Services (AWS). The platform uses Mercurial SCM as its distributed version control engine to track configuration revisions and baseline tags. Hosted on an Amazon EC2 instance within an isolated Virtual Private Cloud (VPC), the platform pairs a React frontend with a FastAPI backend, utilizing an Amazon RDS (PostgreSQL) database for structured metadata. A custom Python deployment engine applies approved configurations directly to target EC2 instances. To maintain continuous compliance affordably, Amazon CloudWatch triggers AWS Lambda functions on a schedule to perform automated drift detection by comparing target server file hashes against approved database baselines. Protected by AWS IAM role-based access control, CloudConfig Ops delivers an auditable and reliable configuration pipeline.

5. Proposed Cloud-Based Solution

Complete Architecture Diagram - Workflow design



Description about the Diagram

The architecture operates within a secure AWS Virtual Private Cloud (VPC). Users access the platform via a React frontend, authenticated by AWS IAM policies. The core logic is hosted on a central EC2 instance. This control plane runs the FastAPI backend and the Mercurial SCM engine, which handles file versioning locally. Metadata such as user roles, approval statuses, and historical audit logs is stored in a managed Amazon RDS database in a private subnet. When a change request is approved, the FastAPI backend uses a Python SSH library to push configuration files to the target

EC2 nodes. To monitor for unauthorized manual changes, Amazon CloudWatch triggers an AWS Lambda function hourly. This Lambda securely connects to the target nodes, calculates file hashes, checks them against the RDS database, and alerts the dashboard if drift is detected.

Modulewise description

- **Control Plane Dashboard:** A React-based UI providing real-time views of configuration baselines, pending change requests, and drift alerts.
- **Configuration Item Manager:** Handles lifecycle CRUD operations for configuration files like .env and nginx.conf.
- **Mercurial SCM Engine:** Executes programmatic revision commits, branch creation, and baseline tagging.
- **Change Request & Approval Workflow:** Restricts server updates by requiring designated reviewers to approve change requests.
- **Python Deployment Engine:** Uses Paramiko to establish secure SSH connections to target EC2 nodes and apply approved configurations.
- **Serverless Drift Detection Engine:** Automates compliance audits on a schedule using AWS Lambda and CloudWatch.

Application Screenshots

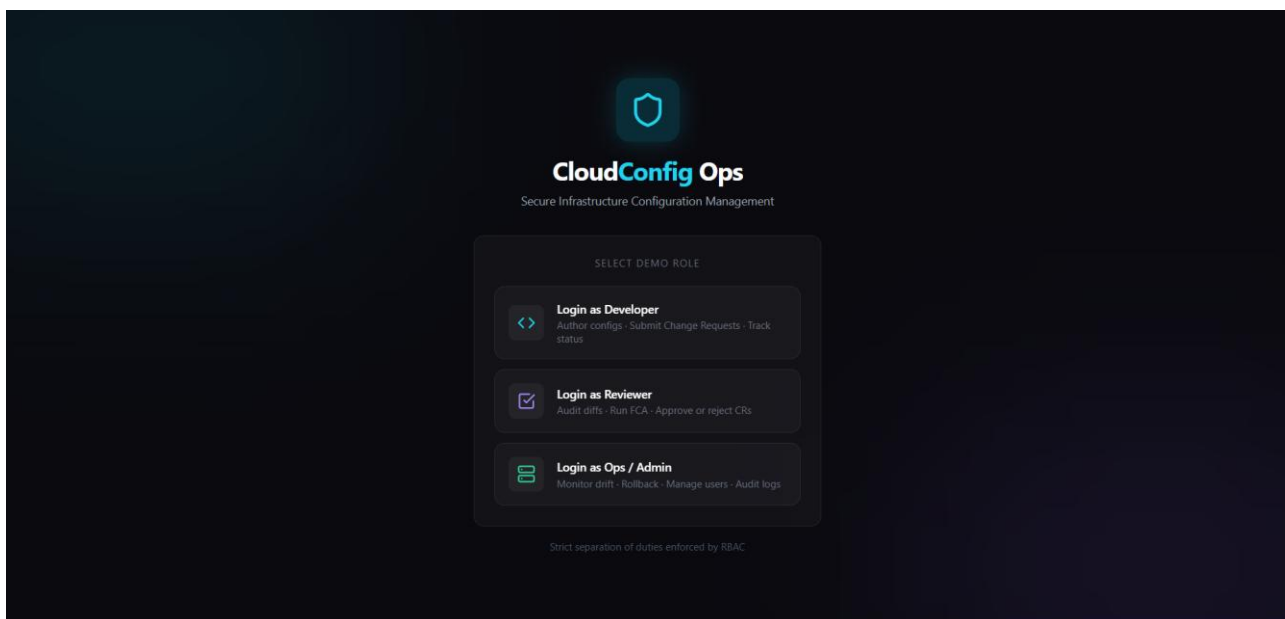


Figure 1: Login page – role selection for Developer, Reviewer and Ops / Admin (role-based access control with separation of duties)

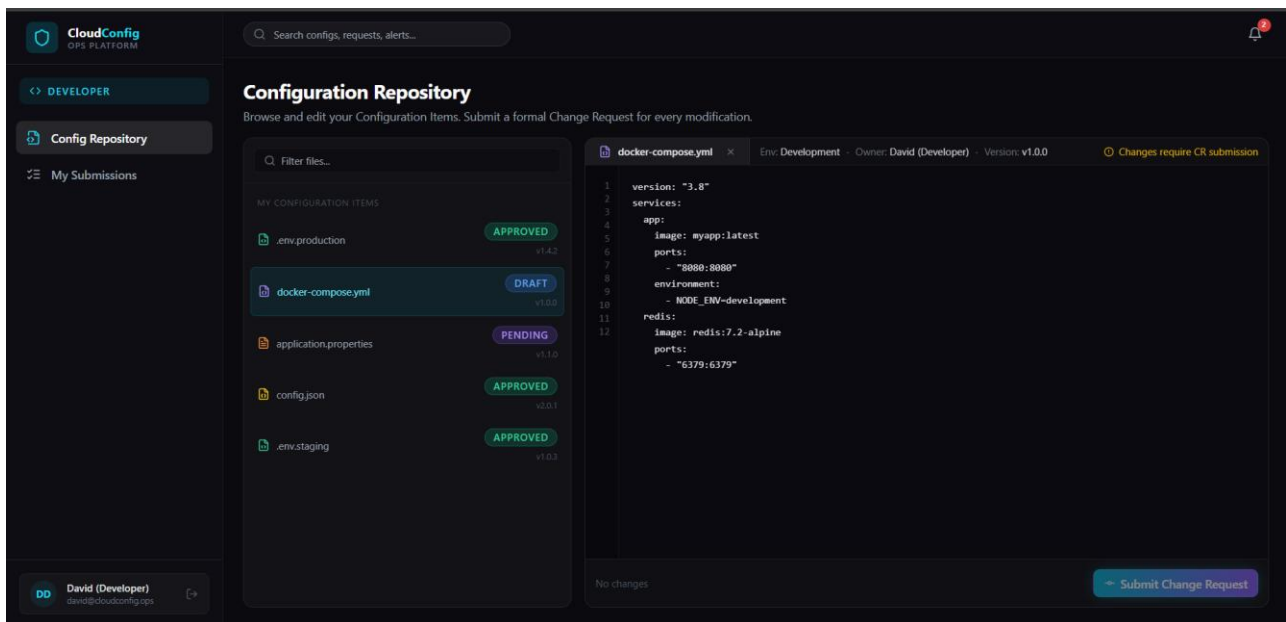


Figure 2: Developer view – Configuration Repository listing configuration items with status and version, and the editor with Submit Change Request

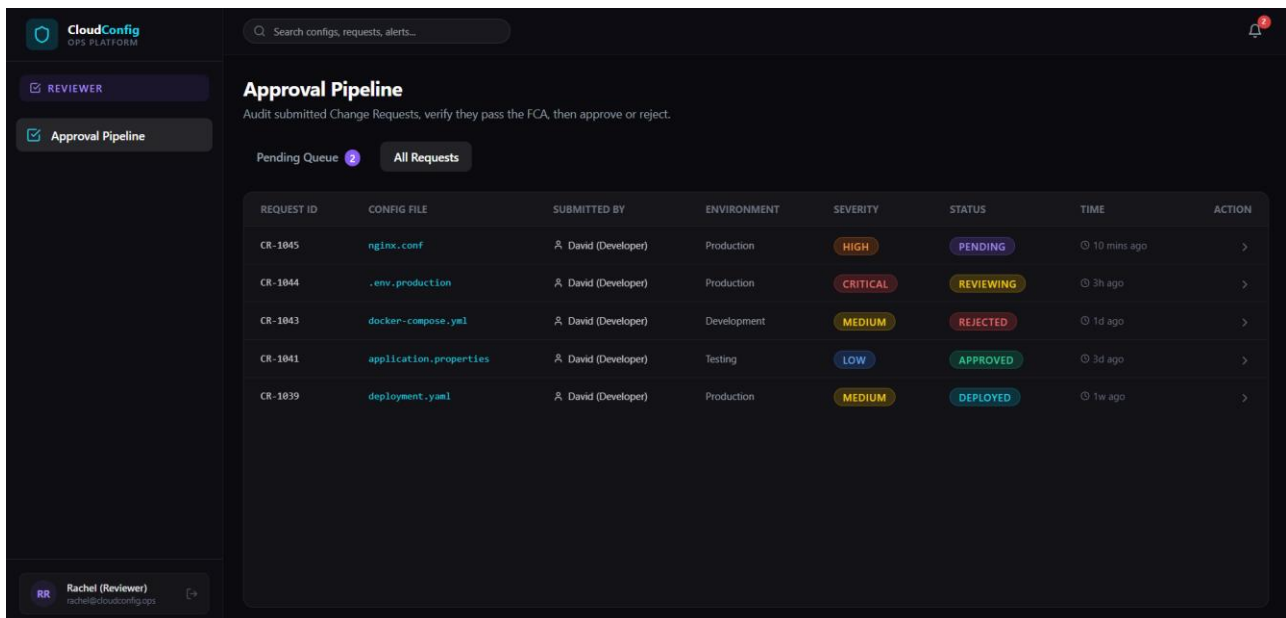


Figure 3: Reviewer view – Approval Pipeline listing change requests with environment, severity and status for audit, approval or rejection

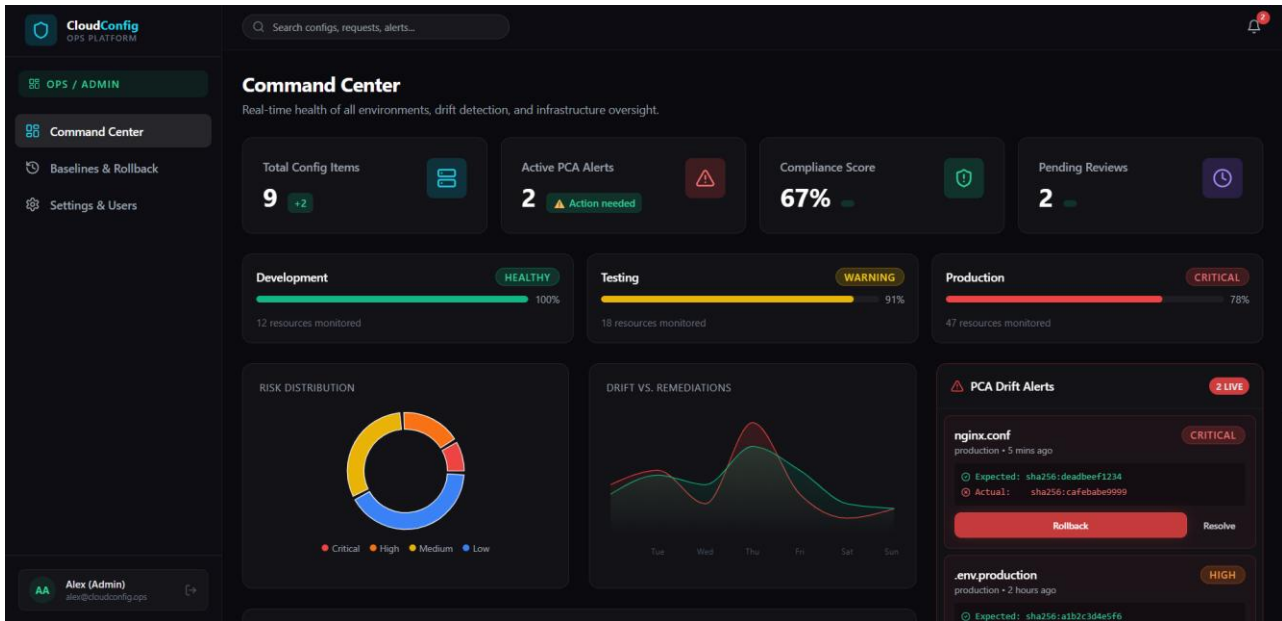


Figure 4: Ops / Admin view – Command Center showing environment health, compliance score, and drift alerts with expected vs. actual hashes and Rollback

6. Functional Requirements

- User Registration & Role-Based Login (Developer, Reviewer, Admin)
- Configuration Item Upload & Editing
- Mercurial Commit & Revision Tracking
- Change Request Submission
- Reviewer Approval / Rejection Pipeline
- Automated Python SSH Deployment execution
- Automated Configuration Drift Detection (Hash comparison)
- Baseline Tagging and One-Click Rollback
- Centralized Deployment Status Dashboard

7. Non-Functional Requirements

- **Scalability:** Database tier scales independently of the compute tier through Amazon RDS.
- **Reliability:** Automated rollback routines recover target nodes from invalid configurations quickly.
- **Security:** Enforced via Amazon VPC network isolation, strict Security Groups, and AWS IAM roles.
- **Availability:** Event-driven serverless architecture ensures auditing mechanisms remain highly available.
- **Performance:** Dashboard data rendered in under 2 seconds; SSH deployments executed within 10 seconds of approval.
- **Cost-effectiveness:** Utilizes auto-stop operational schedules and AWS Lambda to remain strictly within Free Tier billing limits.

8. Requirement Analysis

Hardware Requirements

- Client Machine: Intel i5 / AMD Ryzen 5 processor, 8 GB RAM, 20 GB free storage.

- Cloud Compute Server (Control Plane): AWS EC2 t2.micro or t3.micro (1 vCPU, 1 GB RAM, 7 GB EBS storage).
- Target Cloud Nodes: AWS EC2 t2.micro instances.
- Database Server: Amazon RDS db.t2.micro or db.t3.micro.

Software Requirements

- Operating System: Ubuntu Server 22.04 LTS (AWS EC2 instances).
- Version Control System: Mercurial SCM (v6.x+).
- Backend Framework: Python 3.10+, FastAPI, SQLAlchemy, Paramiko.
- Frontend Framework: React.js, TypeScript, Tailwind CSS.
- Database: PostgreSQL 15.

9. Cloud Services Required

Service	Purpose
Amazon EC2	Acts as the host for the React frontend, FastAPI backend, Mercurial repository, and target deployment servers (Dev, Test, Prod).
Amazon RDS	Stores all structured metadata, including user roles, approval workflows, baseline hashes, and deployment history securely.
AWS Lambda	Runs the custom Python script to perform Drift Detection serverlessly, shutting down instantly after verifying file hashes.
Amazon CloudWatch	Acts as the event scheduler to trigger the Lambda drift-detection function every hour and captures application logs.
AWS IAM	Forms the security backbone by providing specific permissions allowing backend/Lambda to securely access target EC2 instances.
Amazon VPC	Ensures EC2 servers and the RDS database communicate privately while keeping the data layer hidden from the public internet.

AWS Console Screenshots

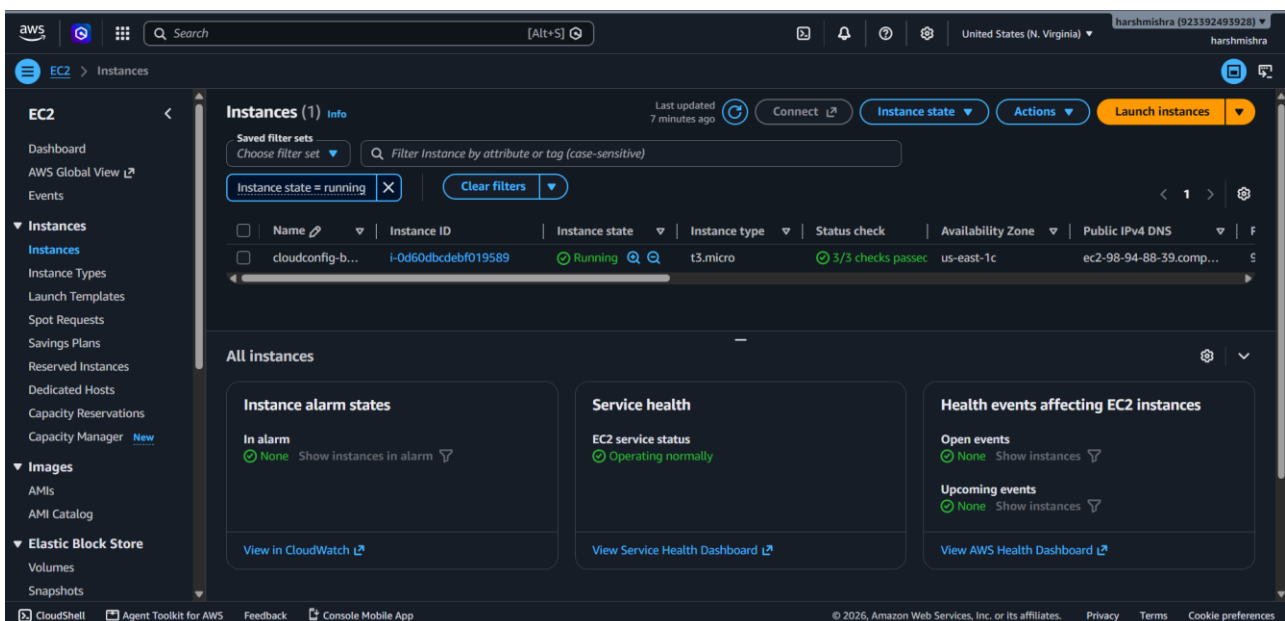


Figure 5: Amazon EC2 – CloudConfig instance (t3.micro) in the running state with 3/3 status checks passed

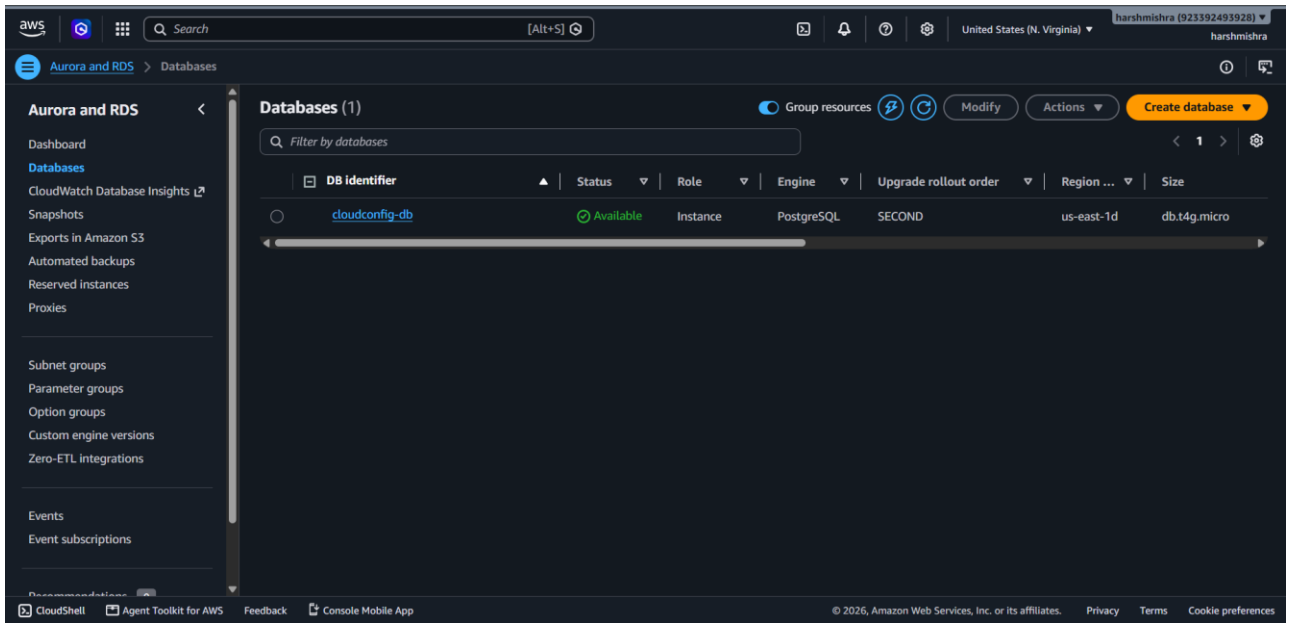


Figure 6: Amazon RDS – cloudconfig-db PostgreSQL instance in the Available state

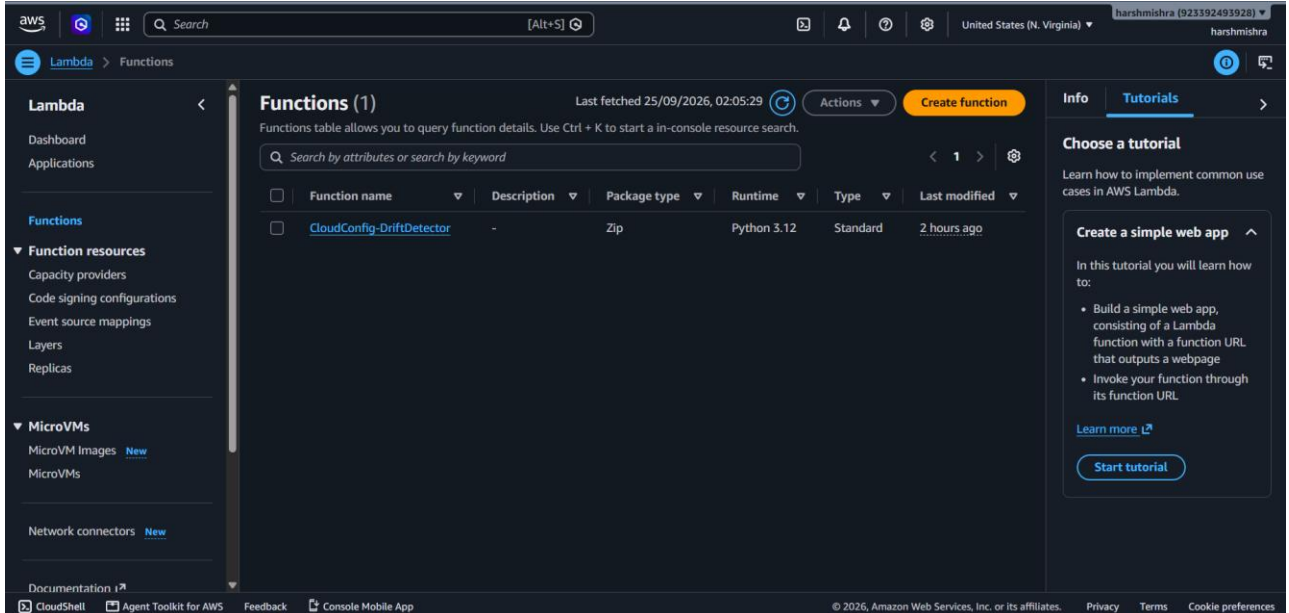


Figure 7: AWS Lambda – CloudConfig-DriftDetector function (Python 3.12) used for serverless drift detection

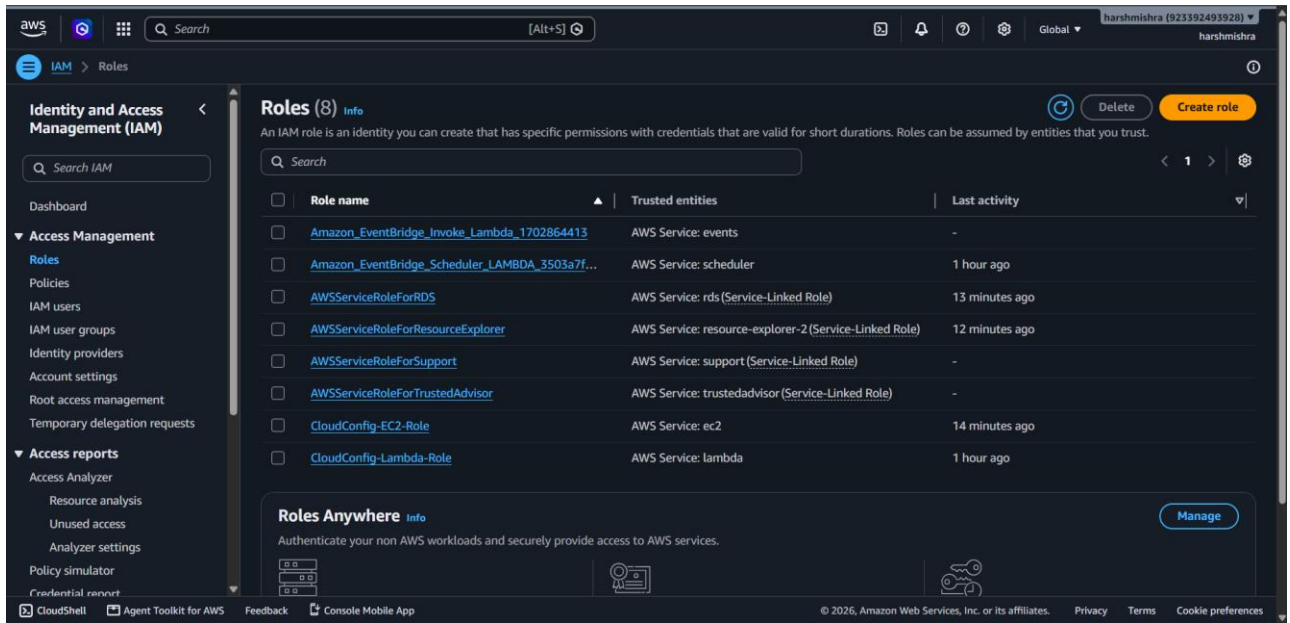


Figure 8: AWS IAM – CloudConfig-EC2-Role and CloudConfig-Lambda-Role with their trusted AWS services

10. References

- [1] Thiagarajan, G., Bist, V., & Nayak, P. (2024). AI-Driven Configuration Drift Detection in Cloud Environments. *International Journal of Communication Networks and Information Security (IJCNIS)*, 16(5), 721–743.
- [2] Harikrishnan, A. (2026). Infrastructure as Code (IaC) Drift Detection: State Management Using Event-Driven Architecture. *Journal of Computational Analysis and Applications (JoCAAA)*, 35(3), 10–23.
- [3] Verdet, A., Hamdaqa, M., Da Silva, L. M. P., & Khomh, F. (2023). Exploring Security Practices in Infrastructure as Code: An Empirical Study. *arXiv:2308.03952 [cs.CR]*.
- [4] Cherukuri, B. R. (2024). Comparative Analysis of Docker and Kubernetes for Cloud Resource and Microservice Management. *International Journal of Science and Research Archive (IJSRA)*, 13(1), 3302–3315.
- [5] Kumar, A., Nadeem, M., & Shameem, M. (2025). A Systematic Literature Review for Investigating DevOps Metrics to Implement in Software Development Organizations. *Journal of Software: Evolution and Process*, 37(1), e2733.
- [6] Babalola, O., Nwatu, C. E., Folorunso, A., & Adewa, A. (2024). A Governance Framework Model for Cloud Computing: Role of AI, Security, Compliance, and Management. *World Journal of Advanced Research and Reviews*, 24(2), 1969–1982.
- [7] Jayaram, Y. (2023). Data Governance and Content Lifecycle Automation in the Cloud for Secure, Compliance-Oriented Data Operations. *International Journal of AI, BigData, Computational and Management Studies*, 4(3), 124–133.
- [8] Olabanji, S. O., Olaniyi, O. O., Adigwe, C. S., Okunleye, O. J., & Oladoyinbo, T. O. (2024). AI for Identity and Access Management (IAM) in the Cloud. *Asian Journal of Research in Computer Science*, 17(3), 38–56.
- [9] Saleh, S. M., Madhavji, N., & Steinbacher, J. (2024). A Systematic Literature Review on Continuous Integration and Deployment (CI/CD) for Secure Cloud Computing. *Proceedings of the 20th International Conference on Web Information Systems and Technologies (WEBIST)*, 331–341.
- [10] Ganesan, P. (2024). Cloud-Based Disaster Recovery: Reducing Risk and Improving Continuity. *Journal of Artificial Intelligence & Cloud Computing*, 3(1), E162.
- [11] Winkler, F., & Westner, M. (2023). A Systematic Literature Review of DevOps Success Factors and Adoption Models. *Proceedings of the 12th International Symposium on Information and Communication Technology (SOICT 2023)*, ACM, 1–8.
- [12] Alharthi, S., Alshamsi, A., Alseiari, A., & Alwarafy, A. (2024). Auto-Scaling Techniques in Cloud Computing: Issues and Research Directions. *Sensors*, 24(17), 5551.
- [13] Deochake, S. (2024). ABACUS: A FinOps Service for Cloud Cost Optimization. *arXiv:2501.14753 [cs.DC]*.
- [14] Essien, I. A., et al. (2023). Integrated Governance, Risk, and Compliance Framework for Multi-Cloud Security and Global Regulatory Alignment. *IRE Journals*, 7(6).
- [15] Nangi, P. R., & Obannagari, C. K. R. (2025). AI-Driven Security Automation for Continuous Compliance Monitoring in Regulated Cloud Environments. *International Journal of Emerging Trends in Computer Science and Information Technology*, 6(2).